

Step 4 – Oracle Verification (EIP-712)

The backend triggers off-chain verification of the flight cancellation, gets a decision, and that decision is cryptographically signed and submitted on-chain — this is the step that actually changes the claim's status from Submitted to Approved (or Rejected).

EIP-712 in this step

Every signature in Step 1 used EIP-191 (a plain string). This step uses **EIP-712** — structured, typed data signing — because the payload being signed here has real internal structure (which claim, a unique request ID, an approve/reject decision, a timestamp) that a plain string wouldn't capture unambiguously, and because EIP-712 signatures are bound to a specific contract + chain via a *domain separator*, so a signature produced for one deployment can never be replayed against a different one.

The actual sequence

1. Backend triggers verification (off-chain, no contract call)

```
POST /claims/1/trigger-verification → oracle_client.request_verification(claim_id=1,
declared_amount=221) → POST http://oracle-service/verify with body {"claimId": 1,
"declaredAmount": 221}.
```

2. Oracle service evaluates the claim (*off-chain, mock logic*)

evaluateClaim({claimId: 1, declaredAmount: 221}) — checks 221 <= 100,000 (the mock approval threshold) → {approved: true, reason: "MOCK: within placeholder threshold"}. Explicitly a placeholder decision function, not real flight-status verification.

3. Oracle service signs the response – EIP-712, step by step

- **Input:** the oracle signer's wallet (private key held by whoever holds the ORACLE_SIGNER_ROLE, address 0x8888...bbbb), and payload { claimId: 1, requestId: 0x1a2b3c... (random 32 bytes), approved: true, timestamp: 1_800_500_100 }.
- **Traversal:**
 1. Builds the domain: { name: "DICS-OracleAdapter", version: "1", chainId: <network id>, verifyingContract: 0x4444...dddd }.
 2. Builds the struct hash: keccak256(abi.encode(ORACLE_RESPONSE_TYPEHASH, 1, 0x1a2b3c..., true, 1_800_500_100)) — where ORACLE_RESPONSE_TYPEHASH is the compile-time constant keccak256("OracleResponse(uint256 claimId,bytes32 requestId,bool approved,uint256 timestamp)").
 3. Computes the final digest: keccak256("\x19\x01" || domainSeparator || structHash) — note the \x19\x01 prefix here versus EIP-191's \x19Ethereum Signed Message:\n<len> prefix from Step 1; this is the actual byte-level difference between the two schemes.
 4. Signs the digest, producing a 65-byte signature.
- **Output:** the signature, e.g. 0x9f8e7d...

4. Oracle service relays it on-chain: `OracleAdapter.submitVerification`

- **Contract:** OracleAdapter at 0x44444444444444444444444444444444DDDD
- **Function:** submitVerification(OracleResponse calldata response, bytes calldata signature)
- **Input:** response = { claimId: 1, requestId: 0x1a2b3c..., approved: true, timestamp: 1_800_500_100 }, signature = 0x9f8e7d...
- **Imported contracts:** EIP712Upgradeable (digest reconstruction), ECDSA (signature

recovery), the locally-declared `IClaimRegistryOracleSink` interface (used in step 5 below)

- **Traversal:**

1. Checks `usedRequestIds[0x1a2b3c...] == false` — this contract's own replay guard, first line of defense.
2. Checks `1_800_500_100 <= block.timestamp` (not from the future) and `block.timestamp <= 1_800_500_100 + responseValidityWindow` (not expired).
3. Recomputes the exact same digest as step 3 above, via `_hashTypedDataV4` (inherited from `EIP712Upgradeable`).
4. Calls `ECDSA.recover(digest, 0x9f8e7d...)` → recovers an address. If the signature matches what was actually produced in step 3, this recovers exactly `0x8888...bbbb`.
5. Checks `hasRole(ORACLE_SIGNER_ROLE, 0x8888...bbbb)` — true.
6. Sets `usedRequestIds[0x1a2b3c...] = true`.
7. Calls `claimRegistry.recordOracleVerification(1, 0x1a2b3c..., true)` — an external call into `ClaimRegistry` (step 5 below).
8. Emits `VerificationSubmitted(1, 0x1a2b3c..., true, 0x8888...bbbb)`.

- **Output:** none (void). **Note who calls this function:** anyone can — submission is permissionless by design; the security guarantee is entirely in the signature, not in who broadcasts the transaction.

5. ClaimRegistry.recordOracleVerification (called from step 4.7)

- **Contract:** ClaimRegistry at 0x33333333333333333333333333333333CCCC
- **Function:** recordOracleVerification(uint256 claimId, bytes32 requestId, bool approved)
- **Input:** claimId = 1, requestId = 0x1a2b3c..., approved = true
- **Caller check:** onlyRole(ORACLE_ROLE) — msg.sender here is 0x4444...dddd (OracleAdapter's own proxy address), which is what must hold ORACLE_ROLE, never a person's key directly.
- **Traversal:**
 1. Checks usedOracleRequestIds[0x1a2b3c...] == false — ClaimRegistry's **own, independent** replay guard, redundant with OracleAdapter's by design (defends in depth).
 2. Sets usedOracleRequestIds[0x1a2b3c...] = true.
 3. Looks up claims[1], confirms it exists.
 4. Sets claims[1].status = Approved, claims[1].processedAt = 1_800_500_150.
 5. Emits ClaimStatusChanged(1, Approved).

Value trace summary

```
claims[1].status: Submitted -> Approved
usedRequestIds[0x1a2b3c...]: false -> true (on OracleAdapter)
usedOracleRequestIds[0x1a2b3c...]: false -> true (on ClaimRegistry, independently)
claims[1].processedAt: 0 -> 1 800 500 150
```

Still no token movement — this step only changes status. Payout is Step 5.